Creating a Simple Spring Boot Application with a REST Endpoint

Activity 2

Problem Explanation

This task asks us to build a simple Spring Boot application using Spring Initializr, add one REST endpoint to it, and then confirm that endpoint actually works by calling it from a browser or from Postman. Spring Boot is a framework built on top of Spring that removes most of the manual setup normally needed to run a Java web application. Instead of configuring a server and wiring components together by hand, Spring Boot comes with sensible defaults already in place, so a working web application can be running within minutes.

Spring Initializr is a web based tool, available at start.spring.io, that generates a ready to use Spring Boot project. It lets you choose the build tool, the Java version, the Spring Boot version, and any starter dependencies your project needs, then packages all of that into a downloadable project folder with the correct structure already set up.

A REST endpoint is simply a URL that a client, such as a browser or Postman, can send a request to, and the application responds with some data, usually in plain text or JSON. For this activity, a single GET endpoint is created that returns a short greeting message, which is enough to prove that the application is running and correctly wired.

Step 1: Generating the Project with Spring Initializr

Go to start.spring.io and set up the project with the following choices:

- Project: Maven
- Language: Java
- Spring Boot version: latest stable version
- Group: com.example
- Artifact: demo
- Packaging: Jar
- Java version: 17 (or whatever version is installed locally)
- Dependencies: Spring Web

Adding the Spring Web dependency is the important part here, since that is what pulls in everything needed to build REST endpoints, including an embedded Tomcat server so the application can run on its own without needing a separately installed server. After filling these in, clicking Generate downloads a zip file containing the project, which can then be extracted and opened in an IDE.

Step 2: Writing the REST Endpoint

Spring Initializr already creates a main application class. A new controller class is added next to it to hold the REST endpoint.

Main Application Class (DemoApplication.java)

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Controller Class (GreetingController.java)

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetingController {

    @GetMapping("/greet")
    public String greet(@RequestParam(value = "name", defaultValue = "Guest") String name) {
        return "Hello, " + name + "! Welcome to your first Spring Boot REST endpoint.";
    }
}
```

Step 3: Running the Application

The application can be started either from the IDE by running the main method inside DemoApplication, or from a terminal in the project folder using the Maven wrapper:

```
./mvnw spring-boot:run
```

Once started, the console will show that the embedded Tomcat server has started, usually on port 8080. The application is then reachable at http://localhost:8080.

Step 4: Testing the Endpoint

Testing in a browser:

Open a browser and visit the following address:

```
http://localhost:8080/greet?name=Kishore
```

The browser will display the plain text response returned by the endpoint.

Testing in Postman:

- Open Postman and create a new request
- Set the method to GET
- Enter the URL: http://localhost:8080/greet?name=Kishore
- Click Send

Postman will show the response body along with the status code, which should be 200 OK if the request was handled successfully.

Expected Output

Visiting the endpoint with a name parameter of Kishore produces this response, shown identically in both the browser and Postman:

```
Hello, Kishore! Welcome to your first Spring Boot REST endpoint.
```

If the endpoint is called without a name parameter, like http://localhost:8080/greet, the default value takes over and the response becomes:

```
Hello, Guest! Welcome to your first Spring Boot REST endpoint.
```

Code Explanation

The DemoApplication class is the entry point of the whole application. The @SpringBootApplication annotation above it is actually a combination of three separate annotations: one that marks the class as a configuration source, one that enables Spring Boot's automatic configuration based on whatever dependencies are on the classpath, and one that tells Spring where to look for other components in the project. The main method simply calls SpringApplication.run, which starts up the embedded server and wires the whole application together.

The GreetingController class is where the actual REST endpoint lives. The @RestController annotation tells Spring that this class handles web requests and that whatever its methods return should be sent directly back in the response body, rather than being treated as the name of a page to render.

The @GetMapping("/greet") annotation connects the greet method to GET requests made to the /greet URL. The @RequestParam annotation on the name parameter tells Spring to pull the value from the query string of the request, and the defaultValue attribute makes that parameter optional, so the endpoint still works even if no name is provided in the request.

When a request comes in, Spring reads the name value from the URL, passes it into the greet method, and the method returns a plain string. Because the class is marked as a REST controller, that returned string becomes the entire body of the HTTP response, which is exactly what shows up in the browser or in Postman.